

FoodAI: AI-Based Android Application

Mini Project

Submitted By

Safa Waseem ——242201054

Course:

Mobile App Development – Theory

Semester:

BSCS 5-B

Assignment # 02:

AI-Based Android Application

Submitted To

Sir Uzair Hassan

Department of Computer Science

KICSIT

October 6, 2026

Contents

1 Introduction **1**

1.1 Problem Statement 1

1.2 Project Objectives 1

1.3 Project Scope 2

2 Embedding Machine-Learning and Deep-Learning Models in Android **2**

2.1 Definition 2

2.2 Training Versus Inference 2

2.3 Why Embed a Model in an Android App? 3

2.4 Benefits of On-Device Inference 3

2.5 Challenges and Limitations 4

2.6 Common Mobile ML Use Cases 4

3 TensorFlow Lite and Mobile Model Deployment **5**

3.1 What Is TensorFlow Lite? 5

3.2 TensorFlow Compared with TensorFlow Lite 5

3.3 Why TensorFlow Lite Is Relevant to FoodAI 5

3.4 Model Optimization 6

4 Step-by-Step Process of Integrating a TFLite Model **6**

4.1 Step 1: Select or Train a Food-Classification Model 6

4.2 Step 2: Convert a Compatible Model to `.tflite` 6

4.3 Step 3: Add the Model and Labels to the Android Project 7

4.4 Step 4: Add a Compatible Runtime Dependency 7

4.5 Step 5: Load the Model 7

4.6 Step 6: Prepare the Image 7

4.7 Step 7: Run Inference 8

4.8 Step 8: Interpret the Output 8

4.9 Step 9: Display the Result and Handle Errors 8

4.10 Step 10: Test and Release Resources 8

5 Overview of the Provided GitHub Repository **9**

5.1 Purpose of Studying the Repository 9

5.2 What to Inspect in the Repository 9

5.3 Repository Cloning Procedure 9

5.4 Common Android Project Structure 10

5.5 Repository Analysis: Input, Processing, and Output 10

5.6 Relevance and Limitations for FoodAI 10

6 Proposed FoodAI Application **11**

6.1 Application Overview 11

6.2 Functional Requirements 11

6.3 Non-Functional Requirements 11

6.4 Scope and Limitations 12

7 Technologies, Tools, Libraries, and AI Model **12**

7.1 Android Studio 12

7.2 Java 12

7.3 XML 12

7.4 Camera and Gallery APIs 12

7.5 TensorFlow Lite Runtime 13

7.6 Food-Classification Model 13

8 System Design, Workflow, and Methodology **13**

8.1 System Workflow 13

8.2 Explanation of Workflow Stages 14

8.3 Implementation Methodology 14

8.4 Security and Privacy Considerations 15

9 Main Features and Functionality **15**

9.1 Splash Screen 15

9.2 Login and Sign-Up Screen 15

9.3 Food Scan Screen 16

9.4 Camera Input 16

9.5 Gallery Input 16
9.6 AI Prediction 16
9.7 Result Screen 16

10 Testing Strategy and Evaluation **16**
10.1 Purpose of Testing 17
10.2 Functional Test Plan 17
10.3 Model Evaluation 17
10.4 Performance Evaluation 18
10.5 Limitations Affecting Results 18
10.6 Screenshots Required for the Final Report 18

11 Expected Outcomes, Limitations, and Future Enhancements **26**
11.1 Expected Outcomes 27
11.2 Current Limitations 27
11.3 Future Enhancements 27

12 Conclusion **27**

1. Introduction

Artificial intelligence has become an important part of modern mobile applications. Smartphones can now run machine-learning models for tasks such as image classification, text recognition, speech processing, and object detection. These capabilities allow applications to provide intelligent features without requiring every operation to be performed on a remote server.

FoodAI is an AI-based Android application designed to identify food items from images. The intended workflow begins with a splash screen, followed by a login or sign-up screen. After entering the application, a user can capture a food image using the camera or select an existing picture from the phone gallery. The selected image is prepared for an image-classification model, and the application displays the predicted food category on a result screen.

The project combines Android application development with machine-learning inference. Android Studio is used to develop and test the application, Java provides application logic, XML defines interface layouts, and TensorFlow Lite is the intended on-device inference technology, provided that a compatible food-classification model is selected and integrated. The model must be suitable for food images and must be used with its correct preprocessing and output interpretation.

FoodAI is a mini-project focused on food image classification. A basic image classifier does not automatically identify every ingredient, estimate calories, determine nutritional values, or provide medical advice. Such capabilities require additional data, models, and validation. The report explains the theory behind embedding ML/DL models in Android, TensorFlow Lite, the repository review process, the proposed FoodAI design, and a realistic testing methodology.

1.1 Problem Statement

Identifying a food item from a photograph may be useful for learning, organizing food images, or demonstrating an AI-powered mobile feature. However, a mobile application cannot classify food correctly merely by opening a camera. It requires a trained model, correctly formatted image input, a reliable inference process, and a mapping from model output to human-readable labels.

The problem addressed by FoodAI is how to combine the Android interface and image-selection workflow with a compatible image-classification model so that a user can submit a food photograph and receive a predicted category.

1.2 Project Objectives

The principal objectives are:

- To design a user-friendly Android interface for the FoodAI application.

- To provide a splash screen and login/sign-up navigation flow.
- To allow the user to capture a new food image or select one from the gallery.
- To understand the role of image preprocessing in machine-learning inference.
- To integrate a compatible food-classification model into an Android application.
- To use TensorFlow Lite or a compatible runtime for on-device inference where supported.
- To display a predicted food label and optionally a meaningful confidence score.
- To study a sample Android/TFLite repository and identify what can and cannot be reused.
- To evaluate the application through documented tests and genuine screenshots.

1.3 Project Scope

The core scope includes interface navigation, camera/gallery image selection, image preparation, model inference, and display of the predicted food class. The scope does not assume calorie estimation, ingredient detection, medical recommendations, cloud-based accounts, or production-grade authentication unless those features are implemented separately.

2. Embedding Machine-Learning and Deep-Learning Models in Android

2.1 Definition

Machine learning is a branch of artificial intelligence in which a system learns patterns from data and uses those patterns to make predictions or decisions. Deep learning is a type of machine learning that uses neural networks with multiple layers to learn complex representations. For image recognition, deep-learning models can learn visual patterns such as edges, textures, shapes, and combinations of features that help distinguish categories.

Embedding a model in Android means including or accessing a trained model from a mobile application and running its inference process as part of the user workflow. A model is generally trained before deployment. The mobile application then uses the trained model to process new input data. This is known as inference, rather than training.

2.2 Training Versus Inference

Training and inference are related but different stages:

- **Training:** A model learns from a dataset. During training, its parameters are adjusted to reduce prediction errors.

- **Validation:** A separate dataset is used to assess model behaviour and help select model settings.
- **Testing:** The model is evaluated on data not used to fit it, giving an estimate of how it may perform on new examples.
- **Inference:** The trained model processes new input and produces an output. In FoodAI, inference means processing a selected food image and producing a predicted class.

Training a deep-learning model can require substantial computation and a large dataset, so it is commonly performed on a computer or cloud environment. A mobile device can then run the trained model for inference if its architecture and resource requirements are suitable.

2.3 Why Embed a Model in an Android App?

Integrating inference into an Android application makes it possible to offer intelligent functionality within the mobile interface. A camera application can classify an object, a translation application can process text, and a food application can predict a food category. The model becomes one part of the overall application rather than a separate tool the user must open.

For FoodAI, the user should not need to understand the technical details of tensors or neural networks. The app provides a simple workflow: select an image, wait for processing, and view the result. The underlying code handles decoding, preprocessing, inference, and output interpretation.

2.4 Benefits of On-Device Inference

- **Reduced network dependence:** If the model and required runtime are stored locally, inference may work without an internet connection.
- **Lower response latency:** Local processing can avoid the delay associated with sending an image to a server and waiting for a response.
- **Privacy benefits:** Images can remain on the device if the application does not transmit them to a server or third-party service.
- **Reduced server costs:** A local inference design may reduce the need for dedicated inference servers.
- **Consistent availability:** The feature may remain available in areas with weak or no network coverage.
- **User convenience:** The camera and gallery can be integrated directly with the prediction screen.

These benefits depend on the implementation. For example, an application may still require

internet access for login, analytics, or downloading resources even if the model itself runs locally.

2.5 Challenges and Limitations

Mobile deployment also creates technical challenges:

- **Memory and storage:** The model and image buffers consume device resources.
- **Processing time:** Large models may respond slowly on low-end devices.
- **Battery usage:** Repeated or intensive inference can consume energy.
- **Input consistency:** Incorrect resizing, image orientation, channel order, or normalization may reduce prediction quality.
- **Device variation:** Hardware, operating-system versions, and available acceleration can differ between devices.
- **Limited class coverage:** A classifier can only reliably distinguish categories represented adequately in its training process.
- **Uncertain predictions:** A model may produce a plausible-looking label for an image that is blurry, unusual, or outside the supported classes.

Therefore, integration should be followed by practical testing on representative images and devices. A model should not be described as accurate or fast without measured evidence.

2.6 Common Mobile ML Use Cases

Use Case	Input	Typical Output
Image classification	A photograph	A category label
Object detection	A photograph or video frame	Object labels and bounding boxes
Text recognition	A document or sign image	Recognized text
Speech recognition	Audio input	Text transcription
Gesture recognition	Camera frames or sensor data	A gesture category
Food recognition	A food photograph	A predicted food category

Table 1: Examples of machine-learning tasks that can be integrated into mobile applications.

3. TensorFlow Lite and Mobile Model Deployment

3.1 What Is TensorFlow Lite?

TensorFlow Lite is Google's technology for deploying machine-learning models on mobile and edge devices. Google's documentation also describes the newer LiteRT ecosystem. The exact runtime and dependency selected for a project should be checked against the model format and current official documentation.

A compatible TensorFlow model can be converted into a `.tflite` model file. An Android application can load that model and run inference through a compatible runtime. The model file contains the model structure and learned parameters needed for inference, while application code prepares inputs and interprets outputs.

3.2 TensorFlow Compared with TensorFlow Lite

Aspect	**Standard TensorFlow**	**TensorFlow Lite**
Main purpose	Model development, training, and general execution	Deployment-oriented inference on mobile and edge devices
Typical formats	SavedModel, Keras and other TensorFlow formats	`.tflite` model format
Android deployment	May be less suitable for lightweight direct mobile inference	Designed for mobile and edge deployment
Optimization	Broad training and execution functionality	Supports deployment-oriented optimizations, including quantization where suitable
Runtime requirements	Depend on the selected model and runtime	May reduce runtime or model overhead depending on model design and conversion

Table 2: General comparison of standard TensorFlow and TensorFlow Lite.

3.3 Why TensorFlow Lite Is Relevant to FoodAI

FoodAI needs to accept a food photograph and return a category on a mobile device. A compatible TensorFlow Lite image-classification model can support this process. On-device inference can reduce network dependence and can improve responsiveness when the model is sufficiently small and the device can run it efficiently.

The technology alone does not supply food-recognition capability. The project still needs a suitable food-classification model, the corresponding class labels, correct preprocessing, compatible input/output tensors, and code to display the result. A model trained for a different

task cannot be converted into a food classifier simply by changing its filename.

3.4 Model Optimization

Model optimization can make inference more practical on mobile hardware. One common technique is quantization, which represents some model values using lower-precision numeric formats. Depending on the model and method, this can reduce model size or improve inference efficiency. However, optimization can also affect prediction quality and must be evaluated.

Other considerations include choosing a model architecture suitable for mobile use, avoiding unnecessarily large image dimensions, and using supported hardware acceleration when appropriate. Optimization decisions should be based on measured performance and compatibility, not on assumptions.

4. Step-by-Step Process of Integrating a TFLite Model

The following workflow describes typical integration stages. Exact code and configuration depend on the selected model, Android project, and runtime version.

4.1 Step 1: Select or Train a Food-Classification Model

The first step is to obtain a model designed to classify food images. The student should record the model name, source, licence, dataset information, supported food classes, expected input format, output format, and evaluation details. If a pre-trained model is used, its documentation should be checked to confirm that it supports the intended food categories.

A generic image model may not recognize food labels unless it has been trained or adapted for that purpose. Similarly, a model created for number guessing cannot be used as a food classifier unless it is replaced by an appropriate model.

4.2 Step 2: Convert a Compatible Model to `.tflite`

For a compatible TensorFlow/Keras model, conversion can be performed in Python. The following example is illustrative and assumes that a trained Keras model already exists.

```
import tensorflow as tf

model = tf.keras.models.load_model("food_model.keras")
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

with open("food_model.tflite", "wb") as file:
    file.write(tflite_model)
```

The filename `food_model.keras` is a placeholder and must be replaced with the actual trained model file. Conversion should be tested to confirm that the model can be loaded and produces the expected output. Conversion does not automatically make an unrelated model suitable for food classification.

4.3 Step 3: Add the Model and Labels to the Android Project

A common approach is to place the model file and label list in the application's assets directory:

```
app/src/main/assets/food_model.tflite
app/src/main/assets/labels.txt
```

The exact path may vary by project configuration. The labels file should contain the class names in the exact order expected by the model output. If the label order does not match the output indices, the app can display an incorrect food name even when the model output is interpreted correctly.

4.4 Step 4: Add a Compatible Runtime Dependency

The app module's Gradle configuration must include a compatible inference runtime. The dependency version should be selected from official documentation and checked against the project's Android Gradle Plugin, minimum Android version, and model requirements. After changing Gradle configuration, the project must sync and build successfully.

The final report should record the actual dependency used in the project rather than listing a version that has not been tested.

4.5 Step 5: Load the Model

The application loads the model from its packaged assets and creates an inference runtime object, such as a TensorFlow Lite Interpreter. Loading should be handled carefully so that a missing file, invalid model, or runtime problem does not crash the application without explanation.

For a production implementation, model loading and inference may need to occur away from the main UI thread if the operation could block interface interaction.

4.6 Step 6: Prepare the Image

The camera or gallery provides an image, but the raw image is not necessarily in the format expected by the model. Preprocessing typically involves:

1. Decoding the selected image into a bitmap or another supported image representation.

2. Correcting image orientation if required.
3. Resizing or cropping to the exact input dimensions.
4. Converting pixel data into the required channel order, often RGB.
5. Applying the normalization or quantization specified by the model.
6. Creating an input tensor or buffer with the required shape and data type.

For example, a model may expect a fixed-size RGB image with floating-point values in a particular numeric range. Another model may expect integer-quantized input. These requirements are model-specific and must be verified rather than guessed.

4.7 Step 7: Run Inference

Once the input has been prepared, the app passes it to the inference runtime. The output buffer must match the model's output shape and type. The runtime processes the input using the trained model and returns the model output. If the tensor dimensions or data type are wrong, inference may fail or return unreliable results.

4.8 Step 8: Interpret the Output

For a classifier, the output may contain one score per class. The application can identify the highest-scoring class and use its index to retrieve the corresponding label. Some models return logits rather than probabilities, and some use quantized outputs; therefore, output interpretation must follow the model's documentation.

A displayed score should not automatically be described as a guaranteed probability. The report should state how the model output is interpreted and avoid unsupported claims about certainty.

4.9 Step 9: Display the Result and Handle Errors

The predicted label is passed to the result screen. A clear interface should show the food category and, where appropriate, a score or short explanation. If the image is unreadable, the model cannot load, or inference fails, the app should display a useful message and allow the user to try again.

4.10 Step 10: Test and Release Resources

Test the complete workflow with multiple supported food classes, different lighting conditions, and images of varying quality. The application should close or release inference resources when they are no longer needed. The test process should include normal operation, cancelled image selection, invalid input, and model-loading failures.

5. Overview of the Provided GitHub Repository

The repository referenced for this assignment is:

https://github.com/Uzair390-Del/NumberGuessingAppTflite

The repository name indicates a TensorFlow Lite number-guessing Android application. It can be studied as a reference for the general structure of an Android project that uses a model-based workflow. However, the repository should not be described as containing FoodAI's food-classification model unless inspection of the actual source and assets confirms that claim.

5.1 Purpose of Studying the Repository

Studying an existing project helps students understand how theory is reflected in practical Android code. The repository may provide examples of Gradle configuration, Android manifest declarations, user-interface code, model asset handling, and inference calls. The exact files and implementation details must be confirmed by opening the repository.

5.2 What to Inspect in the Repository

- **Project configuration:** Identify the Gradle files, Android Gradle Plugin configuration, and module structure.
- **Manifest:** Check the declared activities, app permissions, and launcher activity.
- **User interface:** Identify the screens and layouts used by the app.
- **Model assets:** Determine whether a model file is included and record its actual filename.
- **Model loading:** Locate the code that loads the model and creates the inference runtime.
- **Input handling:** Identify what information the app supplies to the model.
- **Output handling:** Trace how the model output is interpreted and shown to the user.
- **Dependencies:** Record the actual TensorFlow Lite library or task API version, if present.

5.3 Repository Cloning Procedure

Git can be used to download the repository to a local computer. Run the following commands in a terminal:

```
git clone https://github.com/Uzair390-Del/NumberGuessingAppTflite
cd NumberGuessingAppTflite
```

Open the downloaded project folder in Android Studio and wait for Gradle Sync to complete. If the project cannot build due to outdated dependencies or SDK configuration, record the error and resolve it using the actual project settings and official documentation.

5.4 Common Android Project Structure

The following is a general structure to look for. It is not a claim that every listed file exists in the referenced repository.

```
Project/
  settings.gradle
  build.gradle
  app/
    build.gradle
    src/
      main/
        AndroidManifest.xml
        java/ or kotlin/
        res/
          layout/
          drawable/
          values/
        assets/
          model file, if provided
```

5.5 Repository Analysis: Input, Processing, and Output

The repository should be analysed by tracing one complete operation. First, identify what the user enters or selects. Next, locate the code that prepares the input and calls the model. Then, inspect how the output is interpreted and passed to the interface. This end-to-end approach helps distinguish the actual ML logic from ordinary Android interface code.

If the repository uses numeric input or a number-guessing task, its input and output should be described accurately. It should not be presented as image classification simply because it uses TensorFlow Lite.

5.6 Relevance and Limitations for FoodAI

The repository can provide a structural learning reference, but a model trained for number guessing does not meet FoodAI's food-image classification requirement. FoodAI needs a suitable image classifier, the matching labels, correct image preprocessing, and a result screen designed for food categories. Any code reused from the reference must be adapted and tested for the actual

FoodAI requirements.

6. Proposed FoodAI Application

6.1 Application Overview

FoodAI is a proposed Android application that predicts a food category from an image. It combines an accessible interface with a model-inference workflow. The user interacts with the app through screens, while the application code manages navigation, image selection, preprocessing, inference, and result display.

6.2 Functional Requirements

The main functional requirements are:

1. The application should launch through a splash screen.
2. The user should be able to open the login or sign-up interface.
3. The application should provide a food-scanning screen.
4. The user should be able to choose a camera or gallery input method.
5. The selected image should be decoded and prepared for the model.
6. The app should run inference using a compatible food-classification model.
7. The predicted food label should be shown on the result screen.
8. The app should handle cancelled selection and processing errors.

6.3 Non-Functional Requirements

Non-functional requirements describe qualities of the application rather than individual actions:

- **Usability:** The interface should be clear and easy to navigate.
- **Performance:** Inference should complete in a reasonable time on the target device.
- **Reliability:** Common errors should be handled without unexpected crashes.
- **Privacy:** The application should not transmit images without a clear need and appropriate disclosure.
- **Maintainability:** Model handling, image processing, and UI logic should be organized so they can be updated independently.

- **Compatibility:** The selected libraries and APIs should be compatible with the supported Android versions.

6.4 Scope and Limitations

The core feature is image classification. The model predicts among the classes it was designed to recognize; it cannot be expected to identify every possible food item. Mixed dishes, occluded food, poor lighting, unusual camera angles, and images outside the training distribution may lead to incorrect results.

Calorie estimation, nutrition facts, ingredient identification, and diet recommendations are not guaranteed by a food classifier. Those features would need additional suitable models or trusted data sources, clear user communication, and separate validation.

7. Technologies, Tools, Libraries, and AI Model

7.1 Android Studio

Android Studio is the integrated development environment used to create, build, run, and debug Android applications. It provides tools for editing Java and XML files, managing Gradle dependencies, viewing layouts, running emulators, and testing on physical devices.

7.2 Java

Java is used for application logic in the current FoodAI project. Java code can handle screen navigation, button clicks, image selection callbacks, validation, model loading, inference calls, and result display. The exact responsibilities depend on how the application classes are organized.

7.3 XML

XML is used to define Android layouts and visual resources. Layout XML can describe elements such as text views, buttons, image views, containers, and input fields. Keeping layout definitions separate from Java logic can make the interface easier to maintain.

7.4 Camera and Gallery APIs

The camera workflow allows the user to capture a new photograph, while the gallery workflow allows selection of an existing image. Modern Android image-selection options may reduce the need for broad storage permissions. The exact API should be selected according to the app's supported Android versions and implementation requirements.

7.5 TensorFlow Lite Runtime

The runtime is responsible for loading a compatible model and executing inference. The application must prepare the input and interpret the output according to the model specification. The exact dependency, version, model format, and API should be documented from the implemented project.

7.6 Food-Classification Model

The model is the central AI component. The final report should record:

- The model name and source.
- The model licence and permitted use.
- The training dataset or documented training source, where available.
- The supported food classes and label order.
- The expected input dimensions, data type, and normalization.
- The output tensor format and interpretation method.
- Any available evaluation information and known limitations.

If these details have not yet been selected or verified, they should be marked as pending rather than invented.

8. System Design, Workflow, and Methodology

8.1 System Workflow

The conceptual FoodAI workflow is:

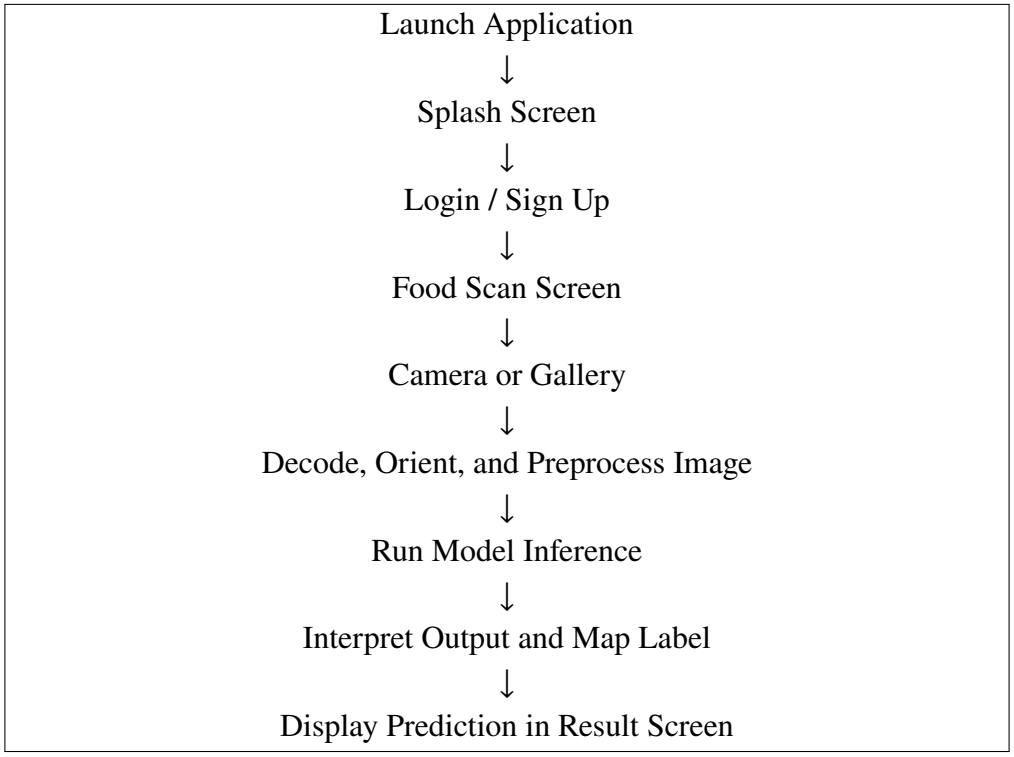


Figure 1: Conceptual workflow of the FoodAI application.

8.2 Explanation of Workflow Stages

1. **Launch:** Android starts the configured launcher activity.
2. **Splash screen:** The app presents its name and branding before navigating onward.
3. **Login/sign-up:** The user enters the application's main flow. The security level depends on the authentication mechanism actually implemented.
4. **Scan screen:** The user chooses whether to capture a new image or select an existing one.
5. **Image preparation:** The image is decoded, oriented, resized, and converted to the model's required input format.
6. **Inference:** The model processes the prepared input and returns its output tensor.
7. **Output interpretation:** Application code maps the model output to the appropriate class label.
8. **Result display:** The predicted category is shown to the user, with a helpful error message if processing fails.

8.3 Implementation Methodology

The application should be developed incrementally:

1. Build and test the splash screen and login/sign-up navigation.
2. Create the scan interface and test camera/gallery selection.
3. Select a compatible food-classification model and record its requirements.
4. Add the model and compatible runtime dependency to the project.
5. Implement image preprocessing according to the model documentation.
6. Implement inference and validate output-to-label mapping.
7. Build the result screen and error messages.
8. Test on an emulator or physical device.
9. Capture real screenshots and record genuine test observations.

8.4 Security and Privacy Considerations

A classroom login screen may demonstrate navigation and input validation without providing real account security. If the prototype stores user details locally, it should not be described as a secure online authentication system. Real passwords should not be stored in plaintext. A production application should use an appropriate secure authentication design.

For privacy, the app should request only permissions it needs, explain why a permission is required, and avoid transmitting food images unless the feature genuinely needs it. If image processing occurs locally, the application should accurately communicate that behaviour.

9. Main Features and Functionality

9.1 Splash Screen

The splash screen introduces FoodAI and displays its branding when the app starts. It may show the application name and a food-related visual theme. After a short transition, it navigates to the next screen. The transition should be tested to ensure that the next activity opens correctly and that repeated navigation does not create an unexpected screen stack.

9.2 Login and Sign-Up Screen

The login/sign-up screen provides fields and buttons for the user flow. Input validation can check whether required fields are empty and whether an email field follows a reasonable format. The exact behaviour depends on the actual code. A demonstration-only login should be described as a prototype rather than as production-grade authentication.

9.3 Food Scan Screen

The scan screen provides the main entry point for food recognition. It can contain camera and gallery buttons, an image preview, and a control to start classification. The interface should make it clear which image will be processed. If a user cancels image selection, the app should return to a stable state without crashing.

9.4 Camera Input

The camera option lets the user capture a new food photograph. Depending on the implementation, the application may use a camera intent or a camera API. The image must be returned to the app and prepared for inference. The implementation should account for permission handling, image orientation, and cases where no image is returned.

9.5 Gallery Input

The gallery option lets the user select an existing photograph. The selected image may have a different size, aspect ratio, or orientation from a camera image. The application should decode it safely, prepare a preview, and apply the same model-specific preprocessing used for camera images.

9.6 AI Prediction

The AI prediction stage uses the prepared image as input to the model. The model returns output values that the application maps to class labels. The result depends on model quality, supported classes, image quality, and preprocessing. The application should not claim a correct identification when the model output is uncertain or the image is outside the model's intended scope.

9.7 Result Screen

The result screen displays the predicted food category in a clear form. It may also show the selected image and a confidence score if the score is correctly interpreted. A retry action can allow users to select another image. The screen should avoid unsupported claims about ingredients, calories, or nutrition.

10. Testing Strategy and Evaluation

10.1 Purpose of Testing

Testing checks whether the application behaves as intended and identifies errors before submission. FoodAI should be tested at both the user-interface level and the model-inference level. Interface testing checks navigation and input selection, while model testing checks whether the preprocessing and prediction pipeline work with supported images.

10.2 Functional Test Plan

The following table is a test plan. It is not evidence that the tests have already passed. Fill in the actual observation after running the application.

Test Case	**Procedure**	**Expected Behaviour / Observation**
Launch	Open the app	Splash screen appears and navigation works
Login validation	Submit empty or invalid fields	Appropriate validation message appears
Camera input	Capture a supported food image	Image is returned and shown in preview
Gallery input	Select an existing image	Image loads with correct orientation
Supported food	Test an image from a supported class	Record predicted label and whether it matches
Blurry image	Test an unclear image	Record prediction and any error/uncertainty handling
Cancelled selection	Open and cancel the picker	App returns to a usable screen
Missing model	Rename the model asset during development	App reports a handled error rather than crashing
Repeated predictions	Classify several images consecutively	App remains responsive and results update correctly

Table 3: FoodAI functional test plan.

10.3 Model Evaluation

A model's accuracy should be evaluated on a documented set of labelled test images. Accuracy is calculated as the number of correct predictions divided by the total number of evaluated predictions, multiplied by 100.

$$\text{Accuracy (\%)} = \frac{\text{Number of correct predictions}}{\text{Total evaluated predictions}} \times 100$$

For example, if a test set contains 50 images and 40 are classified correctly, the observed accuracy would be $40/50 \times 100 = 80\%$. This is only an arithmetic example and is not a claim about FoodAI's actual performance.

A meaningful evaluation should report:

- The number of test images and how they were selected.
- The food classes represented in the test set.
- Whether the images were separate from the model's training data, where that information is available.
- The model name/version and preprocessing configuration.
- The device or emulator used for testing.
- The number and types of incorrect predictions.

10.4 Performance Evaluation

Performance can be measured by recording the time between starting a prediction and displaying its result. Measurements should be taken across multiple images and, where possible, repeated to reduce the influence of one unusually fast or slow run. The report should include the device details and should not claim that inference is fast without measurement.

10.5 Limitations Affecting Results

Results can be affected by poor lighting, low image resolution, cluttered backgrounds, partially hidden food, mixed dishes, and categories that are visually similar. Model performance also depends on whether the training data represent the images users are likely to submit. These limitations should be acknowledged when discussing the observed results.

10.6 Screenshots Required for the Final Report

The following screenshots demonstrate the implemented features of the FoodAI Android application.

Mini Project Structure

1. **Android Studio Project Structure**

   Insert a screenshot of the Android Studio project tree showing relevant Java source files, XML layouts, and model or asset files.

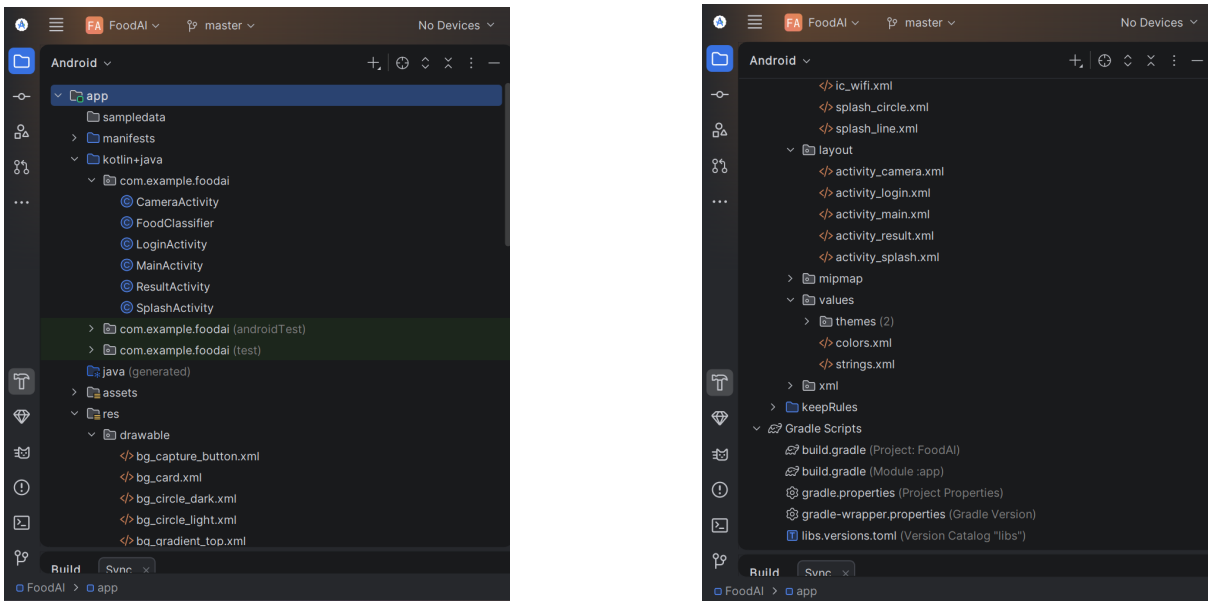


Figure 2: FoodAI Project Structure in Android Studio

2. **Gradle Configuration**

   Add a screenshot of the Gradle configuration showing the actual TensorFlow Lite or other inference dependency, if integrated.

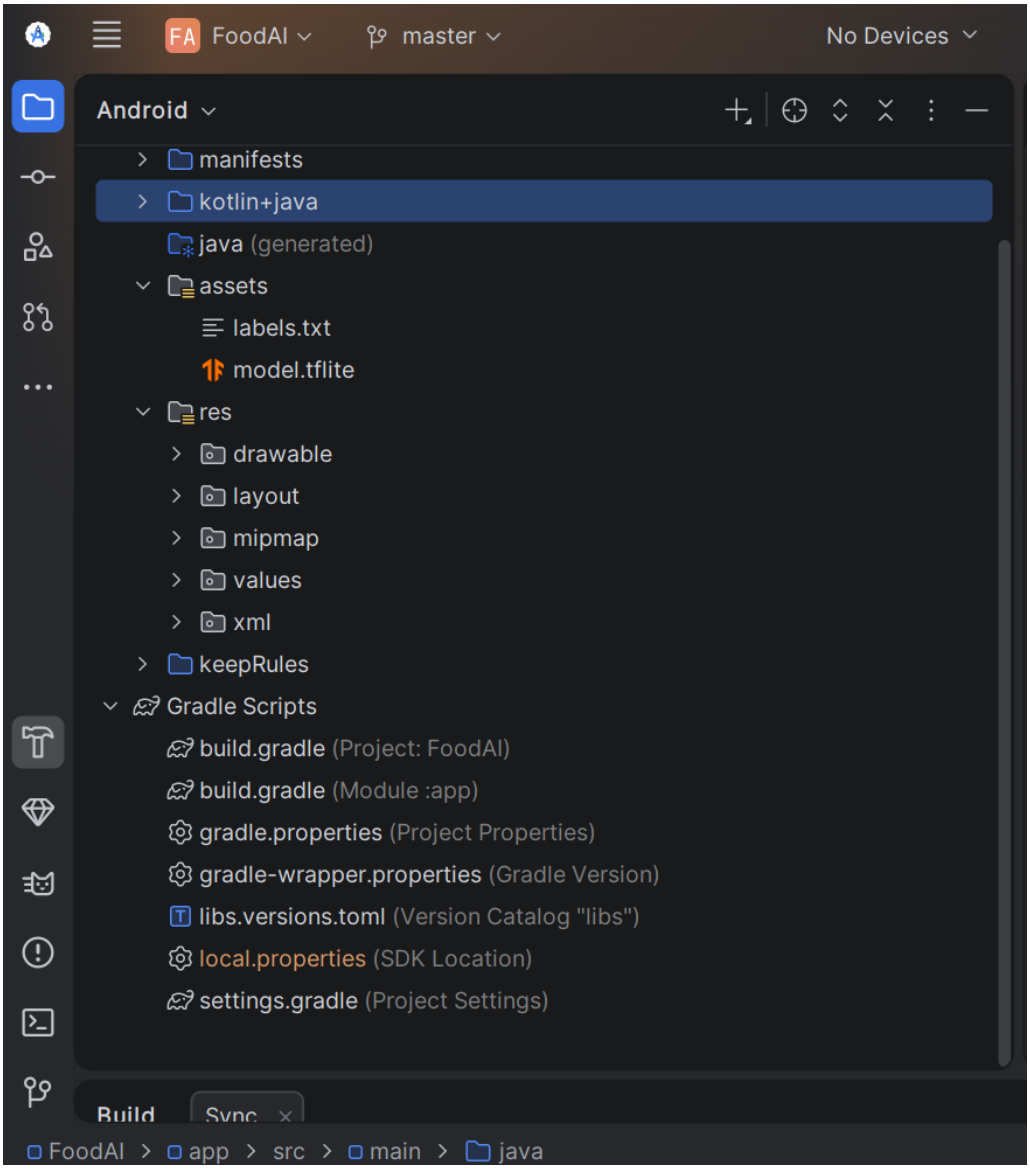


Figure 3: AI Inference Dependency Configuration

3. **Successful Build and Execution**

   Insert a screenshot showing that the application builds and runs successfully in Android Studio or on an Android emulator/device.

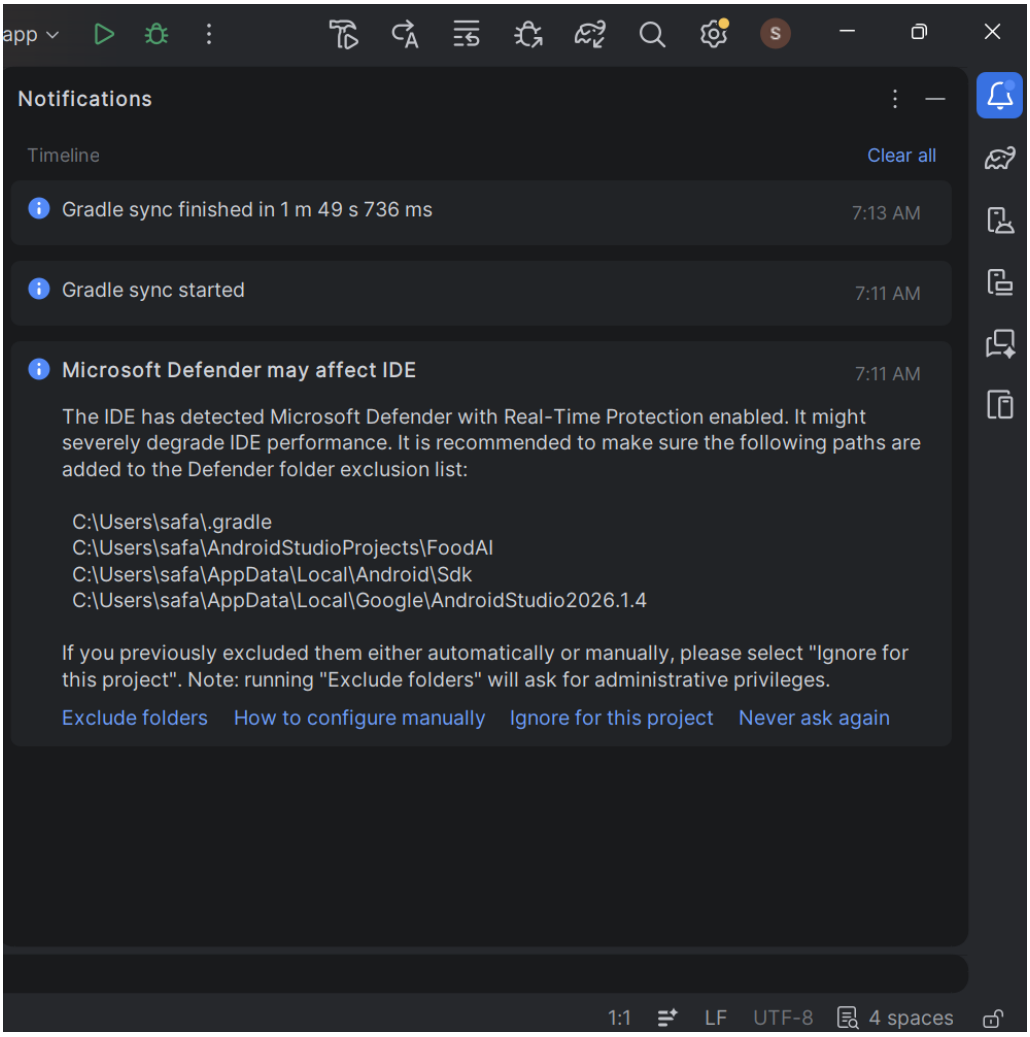


Figure 4: Successful FoodAI Application Execution

FoodAI Application Screenshots

1. **Splash Screen:**

   The splash screen is the first interface displayed when FoodAI starts. It introduces the application and provides a starting point for the user journey.

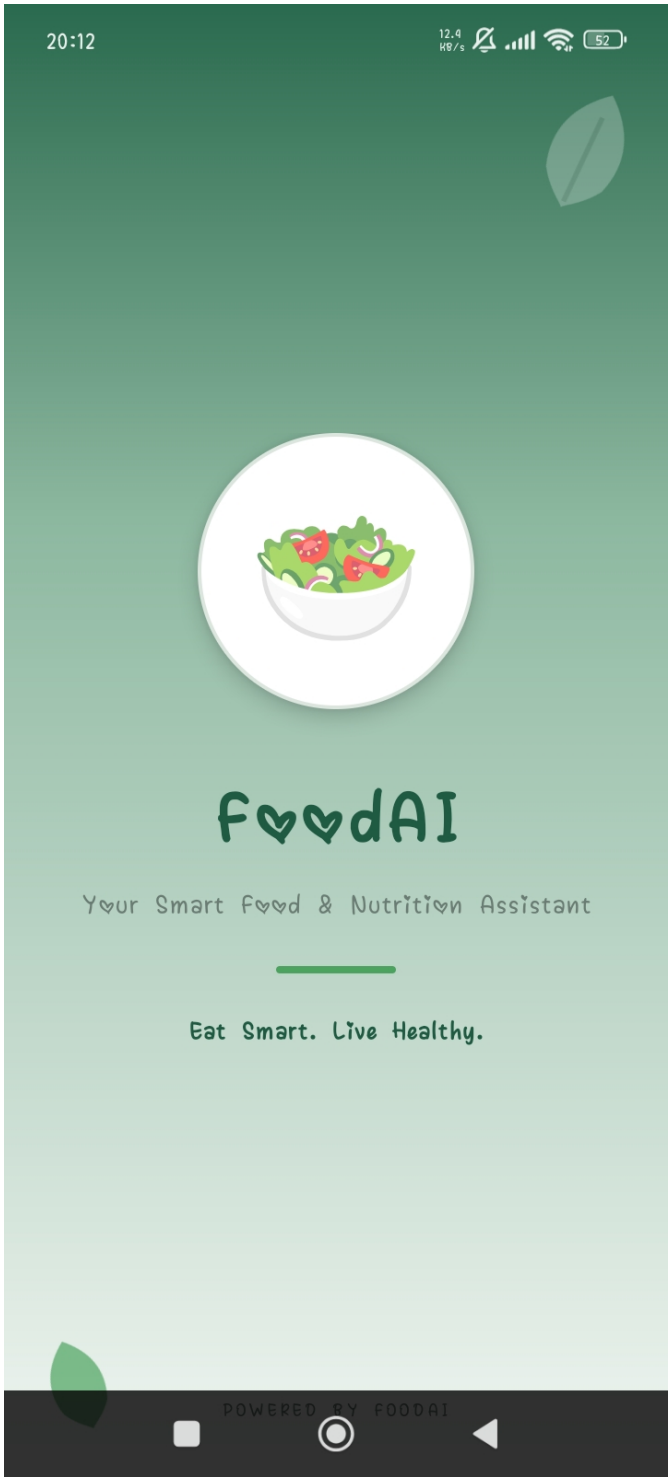


Figure 5: FoodAI Application Splash Screen

2. **Login and Sign-up Screens:**

   The login screen allows existing users to enter their credentials, while the sign-up screen provides an interface for new users to register, if authentication is implemented.

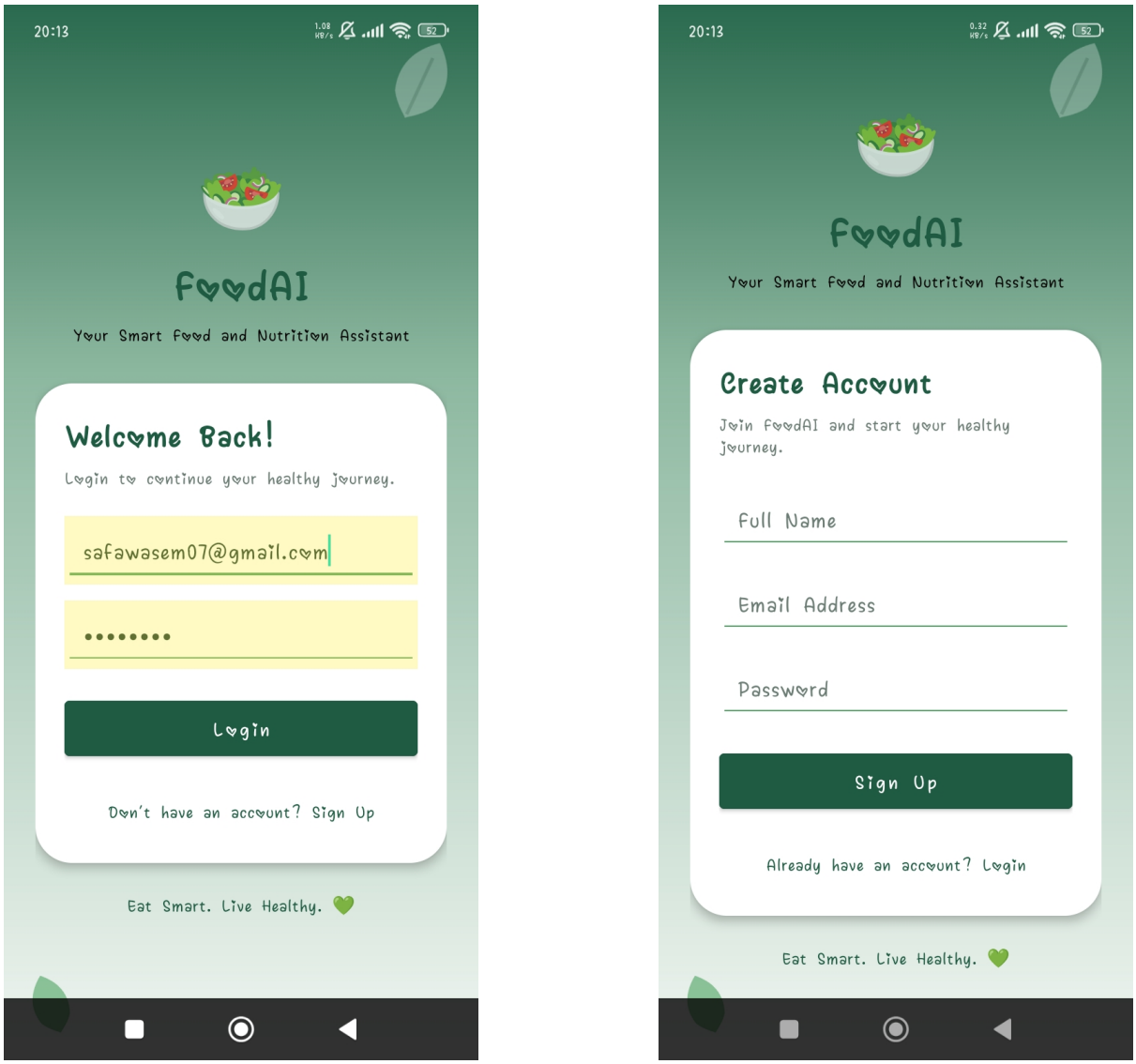


Figure 6: FoodAI Login and Sign-up Interfaces

3. **Home Screen and Navigation to Food Scanning:**

   The home screen introduces the main functionality of FoodAI. When the user clicks the scanning button, the application opens the food-scanning interface.

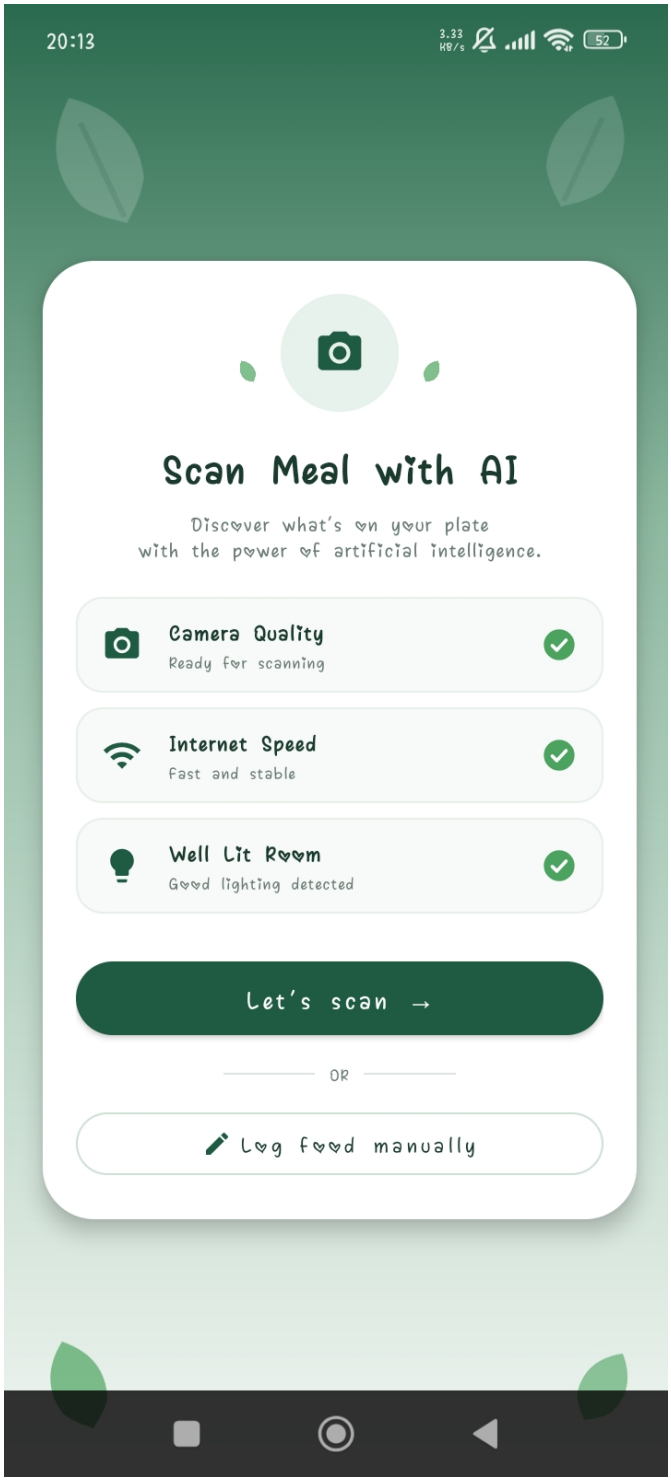


Figure 7: Goto Food Image Scanning Interface

4. **Food-Scanning Screen:**

   This screen allows users to choose a food image from the gallery or capture one using the camera. The selected image is then used for the food recognition process.

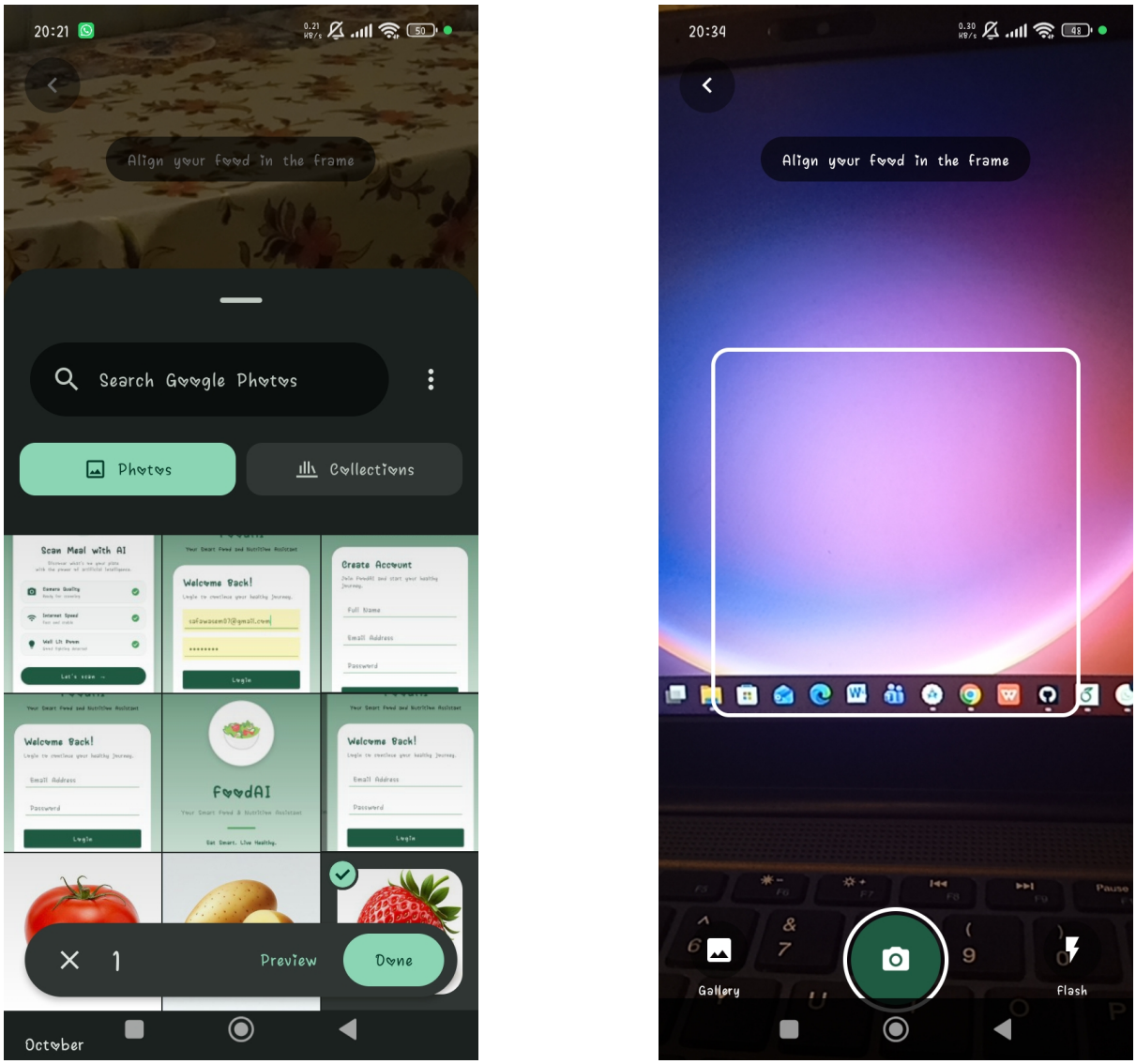


Figure 8: Food Image Selected from Gallery or Camera

5. **Selected or Captured Food Image:**

   The selected or captured image is displayed so the user can review it before analysis. The image may then be prepared according to the input requirements of the AI model.

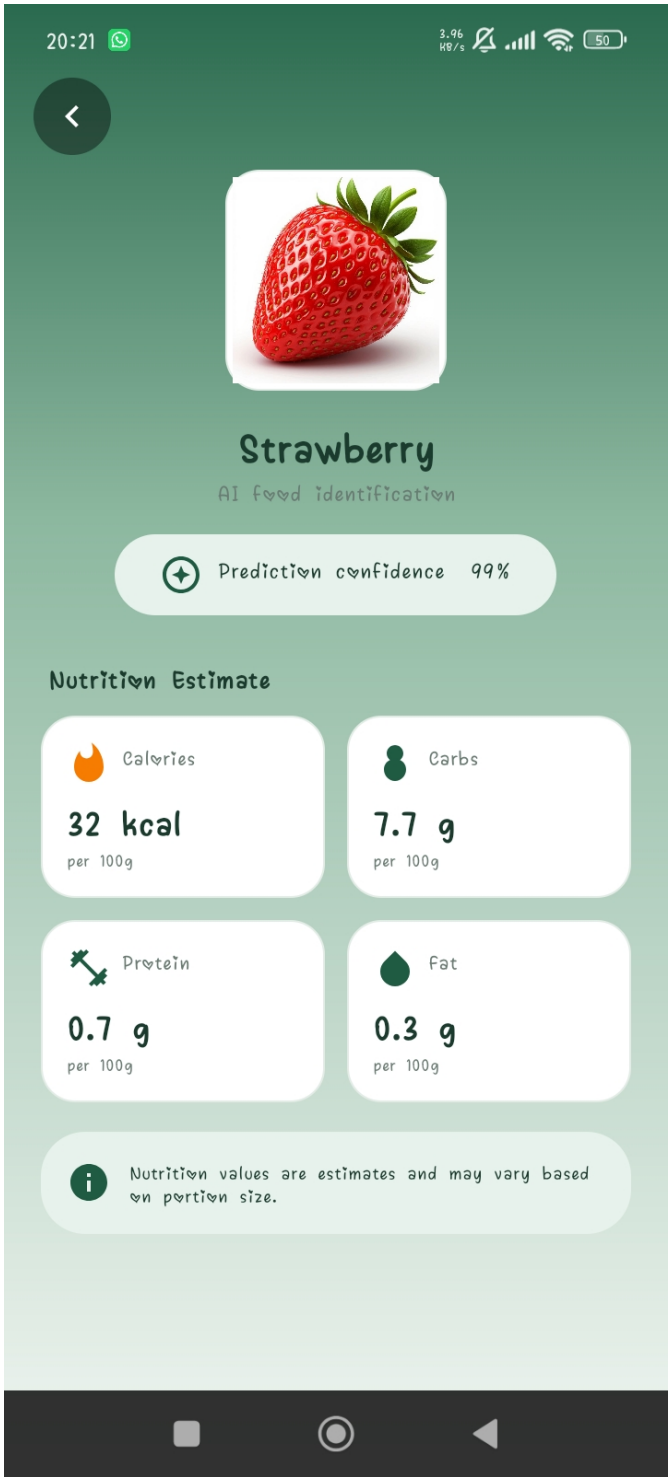


Figure 9: FoodAI AI Prediction Result

11. Expected Outcomes, Limitations, and Future Enhancements

11.1 Expected Outcomes

After a successful implementation, FoodAI should provide a clear user flow from launch to image selection and prediction. A compatible food classifier should process supported images and return labels that the application can display. The project should also demonstrate how Android UI development and on-device model inference can be combined in one application.

These are intended outcomes, not a statement that every feature has already been implemented or tested. The final report should distinguish completed features from planned work.

11.2 Current Limitations

The principal limitations include dependence on the chosen model, limited recognition of unsupported food classes, sensitivity to image quality, and differences in performance across devices. The provided number-guessing repository is not a substitute for a food-classification model. Login and sign-up functionality may also remain a demonstration flow unless a secure authentication service is implemented.

11.3 Future Enhancements

Possible future improvements include:

- Expanding the supported food classes using a suitable dataset and model.
- Improving image preprocessing and handling of different aspect ratios.
- Adding an informative low-confidence or unsupported-image message.
- Measuring inference time and evaluating model size on target devices.
- Improving accessibility, input validation, and error messages.
- Adding nutrition information only through an appropriate, verified data source and with clear limitations.
- Implementing secure authentication if persistent user accounts are required.

Any future enhancement should be added only after its requirements, data sources, and testing approach have been defined.

12. Conclusion

FoodAI is a practical mini-project that combines Android user-interface development with an AI image-classification workflow. The application is intended to guide users through splash and login navigation, camera/gallery image selection, image preprocessing, model inference,

output interpretation, and result display. Each stage contributes to the overall reliability of the application.

TensorFlow Lite can support on-device inference when the model and runtime are compatible. However, successful integration requires more than adding a model file: the model must be appropriate for food images, the input must be prepared according to the model specification, and the output must be mapped to the correct labels. The application must also handle common errors and be tested using real images.

The referenced GitHub repository can help students study the structure of a TensorFlow Lite Android project, but its name indicates a number-guessing task and it should not be represented as FoodAI's food-classification model without evidence. Before submission, the student should add actual project findings, screenshots, model details, and genuine test results. This will make the report technically accurate, transparent, and suitable as a project assignment.

References

[1] Uzair390-Del, *NumberGuessingAppTflite GitHub repository*. Available at: https://github.com/Uzair390-Del/NumberGuessingAppTflite

[2] TensorFlow, *TensorFlow Lite for Android*. Available at: https://www.tensorflow.org/lite/android

[3] TensorFlow, *TensorFlow Lite Android quickstart*. Available at: https://www.tensorflow.org/lite/android/quickstart

[4] Android Developers, *Photo picker*. Available at: https://developer.android.com/training/data-storage/shared/photopicker

[5] Android Developers, *Request runtime permissions*. Available at: https://developer.android.com/training/permissions/requesting